User Documentation for Why3-py v1.0

Akira Tanaka[1] and Yusuke Kawamoto[1]

National Institute of Advanced Industrial Science and Technology (AIST), Tokyo, Japan

Abstract. Why3-py is a Python front-end for the Why3 verification platform that transforms Python programs into verification-oriented WhyML representations suitable for formal verification, addressing the challenges arising from Python's dynamic typing and runtime polymorphism. By using this tool together with StatWhy 1.4, users can verify the correctness of Python statistical programs and identify overlooked assumptions and misuse of statistical analyses.

1 Foreword

The reproducibility crisis in scientific research has drawn increasing attention to the trustworthiness of statistical analyses, particularly meta-analyses that aggregate evidence from multiple studies. However, the correctness of statistical analyses is fundamentally different from that of conventional programs. Specifically, statistical methods crucially rely on assumptions about data-generating processes, such as distributional properties on unobservable true populations, which cannot be verified solely from the program and observed data. To address this issue, we have proposed a formal verification framework for statistical programs, based on explicit specifications of assumptions and interpretations [3,8].

Why3-py is a Python front-end for the Why3 verification platform that transforms Python programs into verification-oriented WhyML representations suitable for formal verification, addressing the challenges arising from Python's dynamic typing and runtime polymorphism. By using this tool together with StatWhy 1.4, users can verify the correctness of Python statistical programs and identify overlooked assumptions and misuse of statistical analyses. To the best of our knowledge, this is the first tool that can formally verify Python code for hypothesis testing and for meta-analyses.

In Sect. 2, we first present how to install the Why3-py tool. In Sect. 3, we present how to use StatWhy 1.4 using Why3-py. In Sect. 4, we show an illustrating example to see how a tool user can verify a statistical program. In Sect. 5, we present more examples to demonstrate how we can verify various hypothesis testing programs and meta-analysis programs written in Python.

1.1 Availability

The source code of the Why3-py tool is publicly available at https://github.com/fm4stats/why3-py including a range of example programs.

1.2 Contact

Report any bugs and requests to https://github.com/fm4stats/why3-py/issues.

1.3 Acknowledgments.

The authors are supported by JSPS KAKENHI Grant Number JP24K02924, Japan. Yusuke Kawamoto is supported by JST, FOREST Grant Number JPMJFR242M, Japan.

2 Installation

In this section, we explain how to install Why3-py together with StatWhy 1.4 [4].

2.1 Why3-py's directory structure.

In Fig. 1, we first show the directory structure of our Why3-py tool.

```
why3-py/
├── README.md ..... README for Why3-py
├── statwhy-install.sh ..... Why3-py installation script
├── statwhy-py ..... Launcher for Why3-py
├── env-why3 ..... Script to define environment variables
├── plugins/ ..... Plugins
│   ├── python/ ..... Why3-py Python plugin
│   └── ..... Other plugins
├── examples/ ..... Examples
│   ├── statwhy-python/ ..... Why3-py examples
│   └── ..... Why3 examples
├── doc-statwhy/ ..... Why3-py documents
└── ..... Other directories and files
```

Fig. 1: Why3-py's directory structure.

2.2 Installing Why3-py

Using the installation script `statwhy-install.sh`, tool users can install Why3-py (together with StatWhy 1.4) and its dependencies (Python, mypy, etc.) into a dedicated directory as follows.

```
$ sudo apt-get install \
  opam \
  wget \
  git \
```

```
  rsync \
  ca-certificates \
  build-essential \
  libsqlite3-dev \
  libssl-dev \
  pkgconf \
  autoconf \
  cvc5 \
  libgmp-dev \
  libcairo2-dev \
  libgtk-3-dev \
  libgtksourceview-3.0-dev
$ git clone --depth 1 --branch statwhy https://github.com/fm4stats/why3-py.git
$ cd why3-py
$ ./statwhy-install.sh $HOME/statwhy statwhy
$ eval $(opam env)
```

In the command `./statwhy-install.sh $HOME/statwhy statwhy`, the path `$HOME/statwhy` specifies the installation directory and `statwhy` specifies the name of the opam switch created by this script.

This software has been tested with Debian GNU/Linux 13 (trixie).

2.3 Remarks on StatWhy

The StatWhy tool [3,4] provides a verification framework for statistical programs using belief Hoare logic (BHL) [5,6] as a theoretical foundation. Given an OCaml/WhyML program annotated with a formal specification written in the Gospel language [1], StatWhy verifies whether a programmer has appropriately annotated the program with the specification, i.e., the requirements for statistical analyses and the interpretation of the test results.

To deal with hypothesis testing, StatWhy provides modules for BHL, real number arithmetic, basic statistics, individual hypothesis testing methods, and so on. StatWhy's implementation is based on the Why3 verification platform [2] and the Cameleer [7] tool using the WhyML language, which integrates programming constructs with formal specifications expressed in terms of preconditions and postconditions.

Remarks on reinstalling StatWhy If you update from an older version of StatWhy or have updated CVC5 since your last StatWhy installation, we recommend deleting the old `~/.statwhy.conf` file. Upon the next launch of StatWhy, a new config file will be generated.

3 Usage

We can verify/execute a Python program *at the* `why3-py` *source directory* using the Why3-py tool.

We can *verify* a Python program via StatWhy by running the following:

```
$ cd why3-py/
$ ./statwhy-py <file-to-be-verified>.py
```

We can *execute* a Python program by running the following:

```
$ cd why3-py/
$ ./env-why3 python3 <file-to-be-verified>.py
```

To debug a Python program, we can use mypy for type checking:

```
$ cd why3-py/
$ ./env-why3 mypy <file-to-be-verified>.py
```

4 Getting Started

After the installation, we can *execute* the first Python example code to compute the combined p-value as follows:

```
$ cd why3-py/
$ ./env-why3 python3 examples/statwhy-python/example0_meta_pv.py
eg_meta_pvs_fisher p-value : 0.115216
```

We remark that `env-why3` must be executed at the Why3-py source directory. The first execution of the tool takes longer time, but subsequent runs can be faster as they reuse cached information.

Next, we can formally *verify* this Python code using the `statwhy-py` launcher.

```
$ ./statwhy-py examples/statwhy-python/example0_meta_pv.py
```

where the `statwhy-py` launcher must also be executed at the Why3-py source directory. Then this launches the IDE of Why3, as shown in Fig. 2.

In the left pane in Fig. 2, there is a single verification condition (VC) to be discharged: `main'vc` (the VC for `Example0_meta_pv`), which is equipped with a question mark. Right-click on this goal and select 'StatWhy' (Not 'CVC5' or the other items) from the context menu, or press '4'. Then StatWhy 1.4 performs the formal verification of the goal. If the prover successfully verifies the goal, all descendants of the goal are folded and a green check mark will appear as shown in Fig. 3.

The Why3-py tool together with StatWhy 1.4 can also be used to identify incorrect requirements in the specification of a python statistical program. This can be seen from the following experiment:

```
$ ./statwhy-py examples/statwhy-python/FAIL_meta_pv.py
```

In the execution, the requirements missing in the specification of this code can be found. Specifically, in Fig. 4, the goal in the right pane represents the verification condition that the prover could not prove. This helps analysts identify the missing requirement, i.e., the assumption that the distribution of p-values is uniformly distributed.

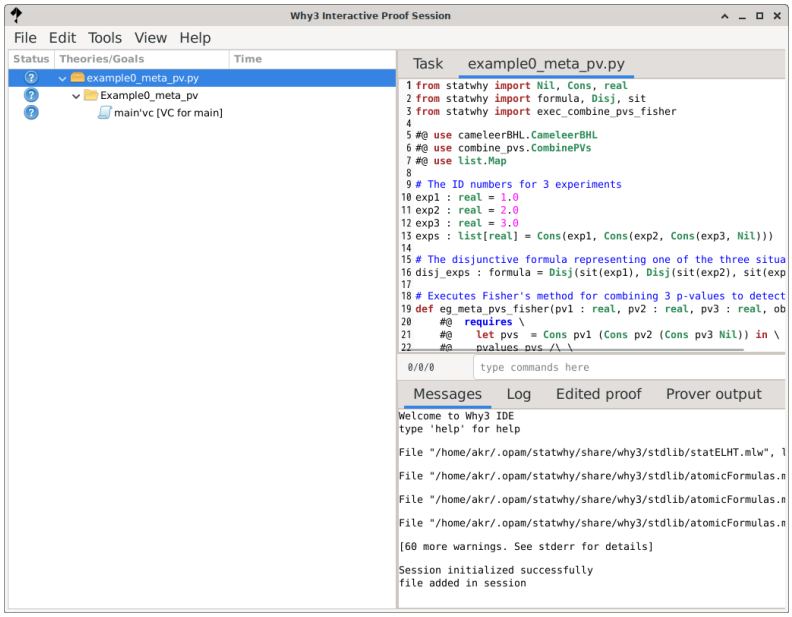

Fig. 2: The Why3 IDE screen launched by our StatWhy 1.4

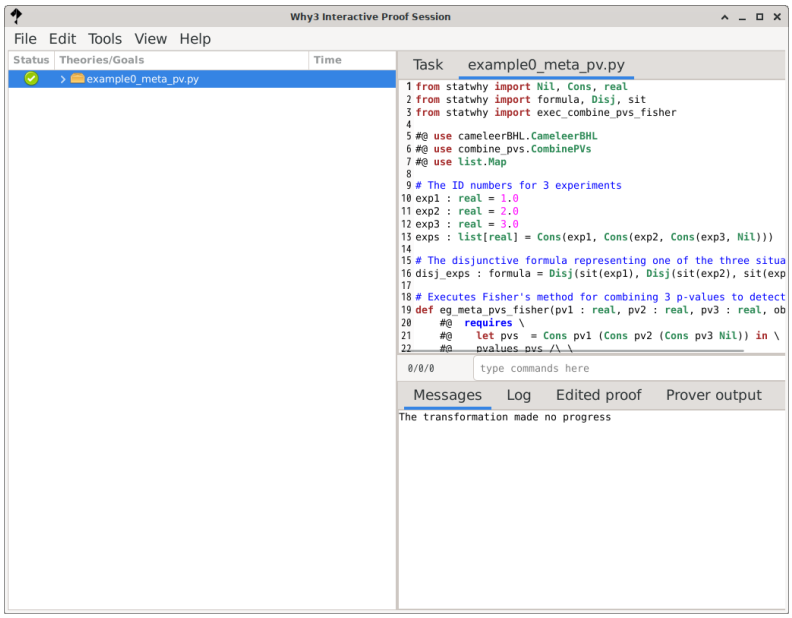

Fig. 3: The Why3 IDE screen showing the success of verification.

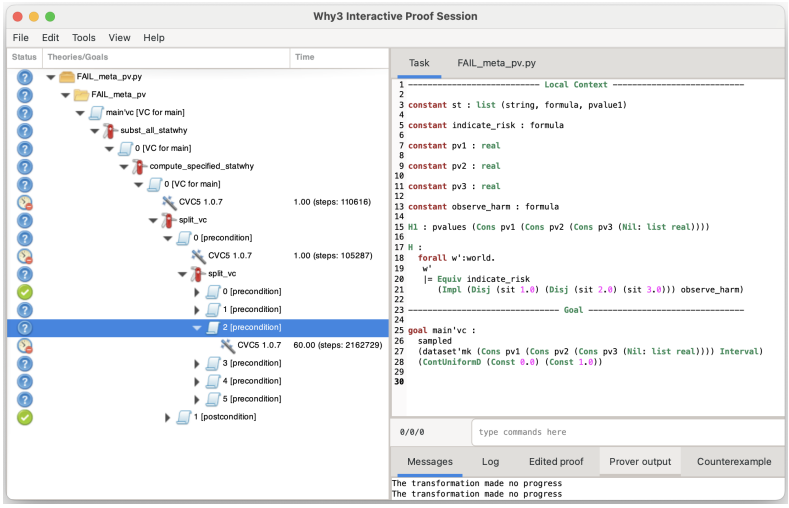

Fig. 4: The Why3 IDE screen showing the *failure* of verification.

5 Case Studies

In this section, we present examples of the formal verification of statistical Python programs annotated with preconditions and postconditions that we want to verify using Why3-py together with StatWhy 1.4. The source code and more examples can be found in Why3-py's directory `examples/statwhy-python/`.

5.1 *t*-test

In Fig. 5, we show `simple_ttest.py`, a python program for one-sample *t*-test. In this program, given a dataset `d` as input, the command `exec_ttest_1samp(pp, 0.0, d, Two)` computes the *p*-value of the one-sample *t*-test with the alternative hypothesis `(mean pp $!= const_term 0.0)` that the mean of the population distribution `pp` (from which the dataset `d` was sampled) is not 0.0.

We can run the program `simple_ttest.py` as follows.

```
$ cd why3-py/
$ ./env-why3 python3 examples/statwhy-python/hypothesis_testing/simple_ttest.py
p-value of ex_ttest_1samp(d1): 0.160664
```

Given this annotated Python code, StatWhy 1.4 together with Why3-py formally verifies the function `ex_ttest_1samp`, which returns a real number given a dataset `d`. We can verify the program `simple_ttest.py` as follows.

```
$ cd why3-py/
$ ./statwhy-py examples/statwhy-python/hypothesis_testing/simple_ttest.py
```

The specification of `ex_ttest_1samp` is written in the Gospel specification language [1]. The comment section starting with `#@` denotes the specification of

```
#@ use cameleerBHL.CameleerBHL
#@ use ttest.Ttest

m = "mean1"
v = "var1"
pp = NormalD(Param(m), Param(v))

def ex_ttest_1samp(d) -> real :
    #@ requires \
    #@    d.scale = Interval /\ \
    #@    is_empty !st /\ \
    #@    sampled d pp /\ \
    #@    (World !st interp) |= Possible (mean pp $< const_term 0.0) /\ \
    #@    (World !st interp) |= Possible (mean pp $> const_term 0.0)
    #@ ensures \
    #@    let p = result in \
    #@    Eq p = compose_pvs (mean pp $!= const_term 0.0) !st && \
    #@    (World !st interp) |= StatB (Eq p) (mean pp $!= const_term 0.0)
    return exec_ttest_1samp(pp, 0.0, d, Two)

#@ execution
d1 = dataset(data=[
    -0.03535040960569304,
    -0.15843246053760296,
    0.8242427720812421,
    -0.42823268167163975,
    0.19617481535793227,
    -0.2707760280674015,
    -1.031181612039124,
    -1.1956445956869606,
    0.09750947857185115,
    -1.1600796427840516
], scale=Interval)

print("p-value of ex_ttest_1samp(d1): %f" % ex_ttest_1samp(d1))
```

Fig. 5: `simple_ttest.py`, Python code that uses the one-sample two-sided t-test.

`ex_ttest_1samp`.[1] More information on the syntax of Gospel can be found on the following webpage: https://ocaml-gospel.github.io/gospel/language/syntax.

At the beginning of the code, the two modules `cameleerBHL.CameleerBHL` and `ttest.Ttest` are imported from StatWhy only for the verification of the program, and not for the execution of the program. The variable `pp` denotes a normal distribution with an unspecified mean and an unspecified variance.

The `requires` clause describes the *precondition* for correctly applying the t-test.

- `d.scale = Interval` specifies that the interval scale is used in this t-test on the dataset `d`. This condition ensures that a dataset with the correct scale of measure is applied to the hypothesis test.
- `is_empty !st` specifies that the record `st` of hypothesis tests is empty. This requirement reminds the programmer to check that no hypothesis test has been conducted before this test.
- `sampled d pp` means that the dataset `d` has been sampled from a normal distribution `pp`. The values of `pp`'s parameters (mean and variance) are not

[1] Programmers are required to use the WhyML language to describe specifications.

specified in the specification. This condition prevents the programmer from forgetting to check whether the population follows a normal distribution.
- `(World !st interp) |= Possible (mean pp $< const_term 0.0)` and `(World !st interp) |= Possible (mean pp $> const_term 0.0)` represent that the analyst has a prior belief that the *lower-tail* hypothesis ($\mathsf{mean}(\mathtt{pp}) < 0.0$) may be true, and that the *upper-tail* hypothesis ($\mathsf{mean}(\mathtt{pp}) > 0.0$) may be true. Thanks to this annotation, programmers are reminded to check whether the alternative hypothesis should be two-tailed or one-tailed, and thus whether the t-test command should be two-tailed or one-tailed.

The `ensures` clause describes the *postcondition*, i.e., what the analyst wants to learn from the t-test in the world where the test has been performed.

- `Eq p = compose_pvs (mean pp $!= const_term 0.0) !st` represents that `p` is equal to the p-value obtained by this hypothesis test with the alternative hypothesis `(mean pp $!= const_term 0.0)`.
- `(World !st interp) |= StatB (Eq p) (mean pp $!= const_term 0.0)` represents that the analyst obtains a statistical belief on the alternative hypothesis `(mean pp $!= const_term 0.0)` with the p-value `p`, in the world equipped with the record `st` of all hypothesis tests executed so far. This logical formula employs a statistical belief modality `StatB` introduced in belief Hoare logic (BHL) [5,6].
- The logical connective `&&` represents an *asymmetric conjunction* to control the goal-splitting transformation; the proof task for A `&&` B is split into those for A and $A \to B$.

The code written after `#@ execution` may be run, but will not be verified by StatWhy.

Variants of t-tests under different requirements on the true populations can be found in the Python programs `ex_ttest_ind_neq.py`, `ex_ttest_ind_eq.py`, and `ex_ttest_paired.py` in the same directory.

5.2 Multiple comparison tests

Let A_{φ_0} and A_{φ_1} be two hypothesis tests with alternative hypotheses φ_0 and φ_1, respectively. There are two possible combinations of A_{φ_0} and A_{φ_1}: the *disjunctive combination* $A_{\varphi_0 \vee \varphi_1}$ with the alternative hypothesis $\varphi_0 \vee \varphi_1$, and the *conjunctive combination* $A_{\varphi_0 \wedge \varphi_1}$ with the alternative hypothesis $\varphi_0 \wedge \varphi_1$. StatWhy 1.4 can check whether a Python program correctly calculates the p-value of these combined tests.

***P*-values of disjunctive alternative hypothesis** Assume that the p-values of A_{φ_0} and A_{φ_1} are p_0 and p_1, respectively. The p-value p of the disjunctive test $A_{\varphi_0 \vee \varphi_1}$ satisfies $p \leq p_0 + p_1$, which is called the *Bonferroni correction*. The Python code below defines a procedure, `example_or_or`, that compares a dataset `d1` with datasets `d2` and `d3`, as well as `d2` with `d3`, then calculates the overall p-value.

examples/statwhy-python/hypothesis_testing/example4.py

```
from statwhy import string, NormalD, Param, real, Two, dataset, Interval
from statwhy import exec_ttest_ind_eq

#@ use cameleerBHL.CameleerBHL
#@ use ttest.Ttest

t_n1 = NormalD(Param("mu1"), Param("var"))
t_n2 = NormalD(Param("mu2"), Param("var"))
t_n3 = NormalD(Param("mu3"), Param("var"))

# H1 : (fmlA \/ fmlB) \/ fmlC
def example_or_or(d1, d2, d3) -> real :
    #@ requires \
    #@   let fmlA_l = mean t_n1 $< mean t_n2 in \
    #@   let fmlA_u = mean t_n1 $> mean t_n2 in \
    #@   let fmlA = mean t_n1 $!= mean t_n2 in \
    #@   let fmlB_l = mean t_n1 $< mean t_n3 in \
    #@   let fmlB_u = mean t_n1 $> mean t_n3 in \
    #@   let fmlB = mean t_n1 $!= mean t_n3 in \
    #@   let fmlC_l = mean t_n2 $< mean t_n3 in \
    #@   let fmlC_u = mean t_n2 $> mean t_n3 in \
    #@   let fmlC = mean t_n2 $!= mean t_n3 in \
    #@   let fml_or_or = fmlA $|| fmlB $|| fmlC in \
    #@   let fml_and_or = fmlA $&& fmlB $|| fmlC in \
    #@   let fml_or_and = fmlA $|| fmlB $&& fmlC in \
    #@   let fml_and_and = fmlA $&& fmlB $&& fmlC in \
    #@   is_empty (!st) /\ \
    #@   sampled d1 t_n1 /\ sampled d2 t_n2 /\ sampled d3 t_n3 /\ \
    #@   d1.scale = d2.scale = d3.scale = Interval /\ \
    #@   independent d1 d2 /\ independent d1 d3 /\ independent d2 d3 /\ \
    #@   (World (!st) interp) |= Possible fmlA_l /\ \
    #@   (World (!st) interp) |= Possible fmlA_u /\ \
    #@   (World (!st) interp) |= Possible fmlB_l /\ \
    #@   (World (!st) interp) |= Possible fmlB_u /\ \
    #@   (World (!st) interp) |= Possible fmlC_l /\ \
    #@   (World (!st) interp) |= Possible fmlC_u
    #@ ensures \
    #@   let fmlA_l = mean t_n1 $< mean t_n2 in \
    #@   let fmlA_u = mean t_n1 $> mean t_n2 in \
    #@   let fmlA = mean t_n1 $!= mean t_n2 in \
    #@   let fmlB_l = mean t_n1 $< mean t_n3 in \
    #@   let fmlB_u = mean t_n1 $> mean t_n3 in \
    #@   let fmlB = mean t_n1 $!= mean t_n3 in \
    #@   let fmlC_l = mean t_n2 $< mean t_n3 in \
    #@   let fmlC_u = mean t_n2 $> mean t_n3 in \
    #@   let fmlC = mean t_n2 $!= mean t_n3 in \
    #@   let fml_or_or = fmlA $|| fmlB $|| fmlC in \
    #@   let fml_and_or = fmlA $&& fmlB $|| fmlC in \
    #@   let fml_or_and = fmlA $|| fmlB $&& fmlC in \
    #@   let fml_and_and = fmlA $&& fmlB $&& fmlC in \
    #@   let p = result in \
    #@   (Leq p) = compose_pvs fml_or_or !st && \
    #@   (World !st interp) |= StatB (Leq p) ((((mean t_n1) $!= (mean t_n2)) $|| fmlB) $||
         fmlC)
    p1 = exec_ttest_ind_eq(t_n1, t_n2, d1, d2, Two)
    p2 = exec_ttest_ind_eq(t_n1, t_n3, d1, d3, Two)
    p3 = exec_ttest_ind_eq(t_n2, t_n3, d2, d3, Two)
    return p1 + p2 + p3
...
```

***P*-values of conjunctive hypotheses** To calculate the p-value of a conjunctive hypothesis $\varphi_0 \wedge \varphi_1$, we take the minimum of the p-values of its subformulas φ_0

and φ_1. The Python code below compares each pair of three datasets `d1`, `d2`, and `d3`, but reports the smallest p-value among the three comparisons.

examples/statwhy-python/hypothesis_testing/example4.py

```
from statwhy import string, NormalD, Param, real, Two, dataset, Interval
from statwhy import exec_ttest_ind_eq

#@ use cameleerBHL.CameleerBHL
#@ use ttest.Ttest

t_n1 = NormalD(Param("mu1"), Param("var"))
t_n2 = NormalD(Param("mu2"), Param("var"))
t_n3 = NormalD(Param("mu3"), Param("var"))
...

# H1 : (fmlA /\ fmlB) /\ fmlC
def example_and_and(d1, d2, d3) :
    #@ requires \
    #@   let fmlA_l = mean t_n1 $< mean t_n2 in \
    #@   let fmlA_u = mean t_n1 $> mean t_n2 in \
    #@   let fmlA = mean t_n1 $!= mean t_n2 in \
    #@   let fmlB_l = mean t_n1 $< mean t_n3 in \
    #@   let fmlB_u = mean t_n1 $> mean t_n3 in \
    #@   let fmlB = mean t_n1 $!= mean t_n3 in \
    #@   let fmlC_l = mean t_n2 $< mean t_n3 in \
    #@   let fmlC_u = mean t_n2 $> mean t_n3 in \
    #@   let fmlC = mean t_n2 $!= mean t_n3 in \
    #@   let fml_or_or = fmlA $|| fmlB $|| fmlC in \
    #@   let fml_and_or = fmlA $&& fmlB $|| fmlC in \
    #@   let fml_or_and = fmlA $|| fmlB $&& fmlC in \
    #@   let fml_and_and = fmlA $&& fmlB $&& fmlC in \
    #@   is_empty (!st) /\ \
    #@   sampled d1 t_n1 /\ sampled d2 t_n2 /\ sampled d3 t_n3 /\ \
    #@   d1.scale = d2.scale = d3.scale = Interval /\ \
    #@   independent d1 d2 /\ independent d1 d3 /\ independent d2 d3 /\ \
    #@   (World (!st) interp) |= Possible fmlA_l /\ \
    #@   (World (!st) interp) |= Possible fmlA_u /\ \
    #@   (World (!st) interp) |= Possible fmlB_l /\ \
    #@   (World (!st) interp) |= Possible fmlB_u /\ \
    #@   (World (!st) interp) |= Possible fmlC_l /\ \
    #@   (World (!st) interp) |= Possible fmlC_u
    #@ ensures \
    #@   let fmlA_l = mean t_n1 $< mean t_n2 in \
    #@   let fmlA_u = mean t_n1 $> mean t_n2 in \
    #@   let fmlA = mean t_n1 $!= mean t_n2 in \
    #@   let fmlB_l = mean t_n1 $< mean t_n3 in \
    #@   let fmlB_u = mean t_n1 $> mean t_n3 in \
    #@   let fmlB = mean t_n1 $!= mean t_n3 in \
    #@   let fmlC_l = mean t_n2 $< mean t_n3 in \
    #@   let fmlC_u = mean t_n2 $> mean t_n3 in \
    #@   let fmlC = mean t_n2 $!= mean t_n3 in \
    #@   let fml_or_or = fmlA $|| fmlB $|| fmlC in \
    #@   let fml_and_or = fmlA $&& fmlB $|| fmlC in \
    #@   let fml_or_and = fmlA $|| fmlB $&& fmlC in \
    #@   let fml_and_and = fmlA $&& fmlB $&& fmlC in \
    #@   let p = result in \
    #@   (Leq p) = compose_pvs fml_and_and !st && \
    #@   (World !st interp) |= StatB (Leq p) fml_and_and
    p1 = exec_ttest_ind_eq(t_n1, t_n2, d1, d2, Two)
    p2 = exec_ttest_ind_eq(t_n1, t_n3, d1, d3, Two)
    p3 = exec_ttest_ind_eq(t_n2, t_n3, d2, d3, Two)
    return min(min(p1, p2), p3)

#@ execution
y1 = dataset(data=[1.79641027917486484, ... , -0.371146977565569358], scale=Interval)
y2 = dataset(data=[2.00283861137070396, ... , 1.85563080881194864], scale=Interval)
y3 = dataset(data=[2.22468795346111614, ... , 1.24697412968412613], scale=Interval)
```

```
res = example_and_and(y1, y2, y3)
print("p-value : %f" % res)
```

We can execute this Python code as follows:

```
$ cd why3-py/
$ ./env-why3 python3 examples/statwhy-python/hypothesis_testing/example4.py
p-value : 0.091818
```

We can verify this code as follows:

```
$ ./statwhy-py examples/statwhy-python/hypothesis_testing/example4.py
```

***P*-value hacking** The *p-value hacking* or *data dredging* is a manipulation of hypothesis testing results to obtain a lower p-value than the actual one. The following code, `example5` in `FAIL_example5.py`, is an example of p-value hacking:

examples/statwhy-python/hypothesis_testing/FAIL_example5.py

```
def example5(d1, d2) :
    #@ requires \
    #@   let fmlA_l : formula = mean t_n $< const_term 1.0 in \
    #@   let fmlA_u : formula = mean t_n $> const_term 1.0 in \
    #@   let fmlA   : formula = mean t_n $!= const_term 1.0 in \
    #@   is_empty !st /\ \
    #@   sampled d1 t_n /\ sampled d2 t_n /\ \
    #@   d1.scale = Interval /\ d2.scale = Interval /\ \
    #@   (World !st interp) |= Possible fmlA_l /\ \
    #@   (World !st interp) |= Possible fmlA_u
    #@ ensures \
    #@   let fmlA_l : formula = mean t_n $< const_term 1.0 in \
    #@   let fmlA_u : formula = mean t_n $> const_term 1.0 in \
    #@   let fmlA   : formula = mean t_n $!= const_term 1.0 in \
    #@   let (p1, p2, p) = result in \
    #@   ((Eq p = compose_pvs fmlA !st (* This is incorrect *) \
    #@     && (World !st interp) |= StatB (Eq p) fmlA) && \
    #@    (Leq (p1 +. p2) = compose_pvs fmlA !st (* This is correct *) \
    #@     && (World !st interp) |= StatB (Leq (p1 +. p2)) fmlA))
    p1 = exec_ttest_1samp(t_n, 1.0, d1, Two)
    p2 = exec_ttest_1samp(t_n, 1.0, d2, Two)
    p = min(p1, p2)
    return (p1, p2, p)
```

`example5` is a Python program that performs the t-test for the mean of `t_n` twice, using different datasets `d1` and `d2`. Then it obtains the p-values for each test, `p1` and `p2`, and reports the lower p-value `p` (defined by `min p1 p2`).

The postcondition of this function consists of two main formulas:

```
Eq p = compose_pvs fmlA !st
&& (World !st interp) |= StatB (Eq p) fmlA
```

and

```
Leq (p1 +. p2) = compose_pvs fmlA !st
&& (World !st interp) |= StatB (Leq (p1 +. p2)) fmlA.
```

In the former assertion, `Eq p = compose_pvs fmlA !st` is an incorrect interpretation of the result. In contrast, the latter represents the correct interpretation, i.e., the sum of `p1` and `p2` is the p-value of `fmlA`.

We can perform the verification of this code as follows.

```
$ ./statwhy-py examples/statwhy-python/hypothesis_testing/FAIL_example5.py
```

Then StatWhy 1.4 proves the correctness of the latter post-condition, but not the former.

Dunnett's method for multiple comparison For another example, we show the python code for Dunnett's method for multiple comparison. In this program, given three datasets `d1`, `d2`, and `d3` as input, the command `exec_dunnett` computes the p-value of the multiple comparison when comparing each treatment with a single control group's dataset `c`.

examples/statwhy-python/hypothesis_testing/ex_dunnett.py

```
def ex_dunnett3(d1 : dataset[real], d2, d3, c) -> array[real] :
    #@ requires for_all (fun d -> d.scale = Interval) \
    #@                   (Cons d1 (Cons d2 (Cons d3 Nil))) /\ \
    #@          independent_list \
    #@            (Cons d1 (Cons d2 (Cons d3 Nil))) /\ \
    #@          c.scale = Interval /\ \
    #@          !st = Nil /\ \
    #@          for_all2 \
    #@            sampled \
    #@            (Cons d1 (Cons d2 (Cons d3 Nil))) \
    #@            (Cons p1 (Cons p2 (Cons p3 Nil))) /\ \
    #@          sampled c p0 /\ \
    #@          for_all \
    #@            (fun p -> (World !st interp) |= Possible (mean p0 $< mean p) /\ \
    #@                      (World !st interp) |= Possible (mean p0 $> mean p)) \
    #@            (Cons p1 (Cons p2 (Cons p3 Nil)))

    #@  ensures \
    #@    let ps = result in \
    #@    for_all (fun t -> let (i,p) = t in \
    #@               (Eq (ps[i]) = compose_pvs (mean p0 $!= mean p) !st) && \
    #@               (World !st interp |= StatB (Eq (ps[i])) (mean p0 $!= mean p))) \
    #@            (enumerate (Cons p1 (Cons p2 (Cons p3 Nil))) 0)

    return exec_dunnett(Cons(p1, Cons(p2, Cons(p3, Nil))), p0, \
                        Cons(d1, Cons(d2, Cons(d3, Nil))), c, Two)
```

We can execute this Python code as follows:

```
$ cd why3-py/
$ ./env-why3 python3 examples/statwhy-python/hypothesis_testing/ex_dunnett.py
[0.9136680848454065, 0.9904330485242874, 0.6647698653636935]
```

We can verify this code as follows:

```
$ ./statwhy-py examples/statwhy-python/hypothesis_testing/ex_dunnett.py
```

5.3 Meta-analyses for Combining p-values

We present examples of meta-analyses in which analysts combine multiple p-values, each corresponding to evidence from an independent study.

Fisher's method We consider a meta-analysis in which an analyst obtains multiple p-values, each corresponding to evidence from an independent study on the harm of a medicine, and then combines them into a single p-value that captures the accumulated evidence of harm using *Fisher's method*.

examples/statwhy-python/example0_meta_pv.py

```
def eg_meta_pvs_fisher(pv1: real, pv2 : real, pv3 : real, observe_harm: formula) -> real :
  #@  requires \
  #@    let pvs = Cons pv1 (Cons pv2 (Cons pv3 Nil)) in \
  #@    pvalues pvs /\ intend_to_detect_excess_of_small_pv /\ \
  #@    forall w': world. (w' |= (Equiv indicate_risk (Impl disj_exps observe_harm))) /\ \
  #@    let d: dataset real = { data = pvs; scale = Interval } in \
  #@    sampled d uniform_pv /\ \
  #@    for_all2 (fun pv exp -> (exists w': world.
  #@                              w' |= StatB (Eq pv) (Impl (sit exp) observe_harm))) pvs exps
  #@  ensures \
  #@    pvalue result /\ \
  #@    (World !st interp |= StatB (Eq result) indicate_risk)
  pvs = Cons(pv1, Cons(pv2, Cons(pv3, Nil)))
  return exec_combine_pvs_fisher(pvs, exps, disj_exps, observe_harm)

#@ execution
res = combine_pvs_fisher(0.1, 0.2, 0.3, observe_harm)
```

Fig. 6: `example0_meta_pv.py`, a Python program that uses Fisher's method for computing a meta-analytic p-value `res` from three independent studies with p-values `pv1`, `pv2`, and `pv3`. The formula `disj_exps` is defined as the disjunction of the experimental situations `(sit exp1)`, `(sit exp2)`, and `(sit exp3)` outside the function.

In Fig. 6, we show `example0_meta_pv.py`, a Python program that calls the function `exec_combine_pvs_fisher` for Fisher's method, which uses SciPy's function to compute a meta-analytic p-value `res` from independent p-values `pv1`, `pv2`, and `pv3`. In contrast to the incorrectly annotated Python code `FAIL_meta_pv.py`, the specification of this program includes all necessary assumptions, hence the verification of the code succeeds.

We can execute this Python code as follows:

```
$ cd why3-py/
$ ./env-why3 python3 examples/statwhy-python/example0_meta_pv.py
eg_meta_pvs_fisher p-value : 0.115216
```

We can verify this code by running the following command line:

```
$ ./statwhy-py examples/statwhy-python/example0_meta_pv.py
```

In this program, the precondition consists of the following formulas.

- The formula `pvalues pvs` checks that `pvs` is a list of p-values, i.e., real numbers over $[0, 1]$.
- `intend_to_detect_excess_of_small_pv` indicates that the meta-analyst intends to detect an excess of small p-values, which is suited for Fisher's method.

– The formula `indicate_risk` specifies the analyst's decision rule, which stipulates that observing harm in one of the three experimental situations lead to reporting a risk.
– The formula `sampled d uniform_pv` represents that the three p-values follow the uniform distribution under the null hypothesis.
– A formula of the form `for_all2 (fun pv exp -> ψ) pvs exps` expresses that ψ holds for each pair of a p-value `pv` and an experiment `exp` in the lists `pvs` and `exps`. So, the last formula of the precondition represents that for each experiment `exp`, the analyst acquires the statistical belief that the alternative hypothesis `observe_harm` holds in the experimental situation `(sit exp)` with a p-value `pv` in a possible world `w'`.

The postcondition states that by combining the three p-values using Fisher's method, we obtain a statistical belief that the alternative hypothesis `indicate_risk` holds with the combined p-value `res`.

Stouffer's method. Why3-py together with StatWhy 1.4 supports popular meta-analysis methods for combining multiple p-values. We show the python code for the two-sided Stouffer's method as follows:

examples/statwhy-python/meta/ex_combine_pvs_stouffer.py

```
def ex_combine_pvs_stouffer_Two(pv1: real, pv2 : real, pv3 : real, fml: formula) -> real :
    #@  requires \
    #@    let pvs  = Cons pv1 (Cons pv2 (Cons pv3 Nil)) in \
    #@    pvalues pvs /\ \
    #@    difficult_to_define_appropriate_weights /\ \
    #@    let d : dataset real = { data = pvs; scale = Interval } in \
    #@    sampled d uniform_pv /\ (* Each p-value in d is sampled uniformly & independently.
          *) \
    #@    for_all2 (fun pv exp -> (exists w' : world. w' |= StatB (Eq pv) (Impl (sit exp) fml
          ))) pvs exps

    #@  ensures \
    #@    let result_pv = twice_pvalue result in \
    #@    pvalue result_pv /\ \
    #@    (World !st interp |= StatB (Eq result_pv) (Impl disj_exps fml))

    pvs = Cons(pv1, Cons(pv2, Cons(pv3, Nil)))
    return exec_combine_pvs_stouffer_Two(pvs, exps, disj_exps, fml)
```

We can execute the Python code for two/upper/lower-sided p-values using non-weighted/weighted Stouffer's method as follows:

```
$ cd why3-py/
$ ./env-why3 python3 examples/statwhy-python/meta/ex_combine_pvs_stouffer.py
ex_combine_pvs_stouffer_Two p-value : 0.022141
ex_combine_pvs_stouffer_Up p-value : 0.063185
ex_combine_pvs_stouffer_Low p-value : 0.063185
ex_combine_pvs_weighted_stouffer_Two p-value : 0.036201
ex_combine_pvs_weighted_stouffer_Up p-value : 0.091048
ex_combine_pvs_weighted_stouffer_Low p-value : 0.091048
```

We can verify this code by running the following command line:

```
$ ./statwhy-py examples/statwhy-python/meta/ex_combine_pvs_stouffer.py
```

Mantel-Haenszel's method. Finally, we show the code execute Mantel-Haenszel's method to combine multiple experiments with two-sided p-values.

examples/statwhy-python/meta/ex_combine_pvs_MantelHaenszel.py

```
def ex_Mantel_Haenszel_Two(dist : distribution, ys : list[ctable]) -> real :
    #@ requires \
    #@   is_empty !st /\ \
    #@   independently_sampled ys /\ (* The strata in ys are independent of each other. *) \
    #@   length ys > 0 /\ all_matrix2x2 ys /\ \
    #@   for_all (fun y -> \
    #@            let d = { data = y; scale = Interval } in \
    #@            sampled d dist) ys /\ \
    #@   for_all (fun t -> \
    #@            exists th : ctable.  (World !st interp) |= (odds_ratio th $= odds_ratio t))
          \
    #@           ys /\ (* The odds ratios for all strata are identical. *) \
    #@   for_all (fun th -> \
    #@            ((World !st interp) |= Possible (odds_ratio th $< const_term 1.0) /\ \
    #@             (World !st interp) |= Possible (odds_ratio th $> const_term 1.0))) \
    #@           ys

    #@ ensures \
    #@   let pv = result in \
    #@   pvalue pv /\ \
    #@   let th = \
    #@     match ys with \
    #@     | Cons hd _ -> hd \
    #@     | Nil -> Nil \
    #@     end in \
    #@   Eq pv = compose_pvs (odds_ratio th $!= const_term 1.0) !st && \
    #@   (World !st interp) |= StatB (Eq pv) (odds_ratio th $!= const_term 1.0)

    return exec_Mantel_Haenszel(dist, ys, Two)
```

We can execute the Python code for two/upper/lower-sided p-values:

```
$ cd why3-py/
$ ./env-why3 python3 examples/statwhy-python/meta/ex_combine_pvs_MantelHaenszel.
    py
ex_Mantel_Haenszel_Two p-value : 0.344964
```

We can verify this code by running the following command line:

```
$ ./statwhy-py examples/statwhy-python/meta/ex_combine_pvs_MantelHaenszel.py
```

A Python-Subset Supported by Why3-py

We show the BNF of the Python-subset supported by Why3-py.

$$
\begin{aligned}
\mathit{file} &::= \mathit{decl}^* \\
\mathit{decl} &::= \mathit{py\_import} \mid \mathit{py\_use} \mid \mathit{py\_function} \mid \mathit{stmt} \\
\mathit{py\_import} &::= \texttt{"from"}\ \mathit{identifier}\ \texttt{"import"}\ \mathit{identifier}\ (\texttt{","}\ \mathit{identifier})^*\ \mathit{NEWLINE} \\
\mathit{py\_use} &::= \texttt{"\#@"}\ \texttt{"use"}\ \mathit{qidentifier}\ (\texttt{","}\ \mathit{qidentifier})^* \\
\mathit{py\_function} &::= \texttt{"def"}\ \mathit{identifier}\ \texttt{"("}\ \mathit{params}^?\ \texttt{")"}\ \mathit{return\_type}^?\ \texttt{":"}\ \mathit{NEWLINE} \\
&\qquad \mathit{INDENT}\ \mathit{spec}^*\ \mathit{stmt}^*\ \mathit{DEDENT} \\
\mathit{params} &::= \mathit{param}\ (\texttt{","}\ \mathit{param})^* \\
\mathit{param} &::= \mathit{identifier}\ (\texttt{":"}\ \mathit{py\_type})^? \\
\mathit{return\_type} &::= \texttt{"->"}\ \mathit{py\_type} \\
\mathit{py\_type} &::= \mathit{identifier}\ (\texttt{"["}\ \mathit{py\_type}\ (\texttt{","}\ \mathit{py\_type})^*\ \texttt{"]"})^? \\
&\mid \texttt{"'"}\ \mathit{identifier} \\
\mathit{spec} &::= \texttt{"\#@"}\ \texttt{"requires"}\ \mathit{term}\ \mathit{NEWLINE} \\
&\mid \texttt{"\#@"}\ \texttt{"ensures"}\ \mathit{term}\ \mathit{NEWLINE} \\
&\mid \texttt{"\#@"}\ \texttt{"variant"}\ \mathit{term}\ (\texttt{","}\ \mathit{term})^*\ \mathit{NEWLINE} \\
\mathit{stmt} &::= \mathit{simple\_stmt}\ \mathit{NEWLINE} \\
&\mid \texttt{"if"}\ \mathit{expr}\ \texttt{":"}\ \mathit{suite}\ \mathit{else\_branch} \\
&\mid \texttt{"while"}\ \mathit{expr}\ \texttt{":"}\ \mathit{loop\_body} \\
&\mid \texttt{"for"}\ \mathit{identifier}\ \texttt{"in"}\ \mathit{expr}\ \texttt{":"}\ \mathit{loop\_body} \\
\mathit{else\_branch} &::= \epsilon \mid \texttt{"else:"}\ \mathit{suite} \mid \texttt{"elif"}\ \mathit{expr}\ \texttt{":"}\ \mathit{suite}\ \mathit{else\_branch} \\
\mathit{suite} &::= \mathit{simple\_stmt}\ \mathit{NEWLINE} \\
&\mid \mathit{NEWLINE}\ \mathit{INDENT}\ \mathit{stmt}\ \mathit{stmt}^*\ \mathit{DEDENT} \\
\mathit{simple\_stmt} &::= \mathit{expr} \mid \texttt{"return"}\ \mathit{expr} \\
&\mid \mathit{identifier}\ (\texttt{":"}\ \mathit{py\_type})^?\ \texttt{"="}\ \mathit{expr} \\
&\mid \mathit{identifier}\ \texttt{"["}\ \mathit{expr}\ \texttt{"]"}\ \texttt{"="}\ \mathit{expr} \\
&\mid \texttt{"break"} \mid \texttt{"\#@"}\ \texttt{"label"}\ \mathit{identifier} \\
&\mid \texttt{"\#@"}\ (\texttt{"assert"} \mid \texttt{"assume"} \mid \texttt{"check"})\ \mathit{term} \\
\mathit{loop\_body} &::= \mathit{simple\_stmt}\ \mathit{NEWLINE} \\
&\mid \mathit{NEWLINE}\ \mathit{INDENT}\ \mathit{loop\_annot}^*\ \mathit{stmt}\ \mathit{stmt}^*\ \mathit{DEDENT} \\
\mathit{loop\_annot} &::= \texttt{"\#@"}\ \texttt{"invariant"}\ \mathit{term}\ \mathit{NEWLINE} \\
&\mid \texttt{"\#@"}\ \texttt{"variant"}\ \mathit{term}(\texttt{","}\ \mathit{term})^*\ \mathit{NEWLINE} \\
\mathit{expr} &::= \texttt{"None"} \mid \texttt{"True"} \mid \texttt{"False"} \\
&\mid \mathit{integer\text{-}literal} \mid \mathit{real\text{-}literal} \mid \mathit{string\text{-}literal} \\
&\mid \mathit{identifier} \mid \mathit{identifier}\ \texttt{"["}\ \mathit{expr}\ \texttt{"]"} \mid \texttt{"-"}\ \mathit{expr} \mid \texttt{"not"}\ \mathit{expr} \\
&\mid \mathit{expr}\ \mathit{binop}\ \mathit{expr} \\
&\mid \mathit{identifier}\ \texttt{"("}\ (\mathit{expr}(\texttt{","}\ \mathit{expr})^*)^?\ \texttt{")"} \\
&\mid \mathit{identifier}\ \texttt{"("}\ \mathit{keyword\_item}\ (\texttt{","}\ \mathit{keyword\_item})^*\ \texttt{")"} \\
&\mid \mathit{expr}\ (\texttt{","}\ \mathit{expr})^+ \mid \texttt{"["}\ (\mathit{expr}\ (\texttt{","}\ \mathit{expr})^*)^?\ \texttt{"]"} \\
&\mid \mathit{expr}\ \texttt{"if"}\ \mathit{expr}\ \texttt{"else"}\ \mathit{expr} \\
&\mid \texttt{"("}\ \mathit{expr}\ \texttt{")"} \\
\mathit{binop} &::= \texttt{"+"} \mid \texttt{"-"} \mid \texttt{"*"} \mid \texttt{"//"} \mid \texttt{"\%"} \\
&\mid \texttt{"=="} \mid \texttt{"!="} \mid \texttt{"<"} \mid \texttt{"<="} \mid \texttt{">"} \mid \texttt{">="} \mid \texttt{"and"} \mid \texttt{"or"} \\
\mathit{keyword\_item} &::= \mathit{identifier}\ \texttt{"="}\ \mathit{expr} \\
\mathit{qidentifier} &::= (\mathit{identifier}\ \texttt{"."})^*\ \mathit{identifier}
\end{aligned}
$$

References

1. Charguéraud, A., Filliâtre, J., Lourenço, C., Pereira, M.: GOSPEL - providing ocaml with a formal specification language. In: FM'19. pp. 484–501. Springer (2019)
2. Filliâtre, J., Paskevich, A.: Why3 - where programs meet provers. In: ESOP'13. LNCS, vol. 7792, pp. 125–128. Springer (2013)
3. Kawamoto, Y., Kobayashi, K., Suenaga, K.: StatWhy: Formal verification tool for statistical hypothesis testing programs. In: Proc. CAV 2025, Part II. LNCS, vol. 15932, pp. 216–230. Springer (2025)
4. Kawamoto, Y., Kobayashi, K., Suenaga, K.: User Documentation for StatWhy v1.4 (2026), available at https://github.com/fm4stats/statwhy
5. Kawamoto, Y., Sato, T., Suenaga, K.: Formalizing statistical beliefs in hypothesis testing using program logic. In: Proc. KR'21. pp. 411–421 (2021)
6. Kawamoto, Y., Sato, T., Suenaga, K.: Sound and relatively complete belief Hoare logic for statistical hypothesis testing programs. Artif. Intell. **326**, 104045 (2024)
7. Pereira, M., Ravara, A.: Cameleer: A deductive verification tool for OCaml. In: Proc. CAV 2021, Part II. LNCS, vol. 12760, pp. 677–689. Springer (2021)
8. Tanaka, A., Kawamoto, Y.: Why3-py: A tool for formal verification of hypothesis testing and meta-analysis in Python (2026), https://arxiv.org/pdf/2607.03951